

Day 1: Lab Exercises — Foundations of Parallel Computing & Concurrency

Duration: 4 hours (self-paced practice)

Reference: [day1/README.md](#) and [day1/demo/](#) programs

Note: These exercises are for practice only. Work through them at your own pace — start from Exercise 1 and progress sequentially.

Exercise 1: JSON Configuration Manager

Topic: Data Serialization with JSON

Problem Statement

Write a program `ex01_config_manager.py` that manages application configuration using JSON serialization.

Your program should:

1. Define a Python dictionary representing a parallel computing job configuration:

```
config = {
    "job_name": "matrix_benchmark",
    "num_workers": 4,
    "chunk_size": 1000,
    "timeout_sec": 30.0,
    "retry_on_failure": True,
    "input_files": ["data_part1.csv", "data_part2.csv",
"data_part3.csv"],
    "output_dir": "/results/run_001",
    "parameters": {
        "algorithm": "strassen",
        "precision": "double"
    }
}
```

2. **Save** this configuration to a file called `job_config.json`.
3. **Load** the configuration back from the file.
4. **Modify** a value (e.g., change `num_workers` to 8) and save it again.
5. **Pretty-print** the final configuration to the console.

Hints & Guidelines

- Use `json.dump()` with `indent=4` for human-readable output.
- Use `json.load()` to read the file back into a dictionary.

- After modifying the loaded dictionary, write it back using `json.dump()` again — JSON files are fully overwritten, not appended.
- Verify that the file on disk reflects your changes by reading it one final time.

Best Practices

- Always use context managers (`with open(...) as f:`) for file I/O — they guarantee the file is closed even if an error occurs.
 - Add error handling with `try/except` for `FileNotFoundError` and `json.JSONDecodeError` when loading files.
 - Use meaningful key names — the config file should be self-documenting.
-

Exercise 2: Pickle a Task Queue

Topic: Object Serialization with Pickle

Problem Statement

Write a program `ex02_task_queue_pickle.py` that demonstrates pickling a queue of computational tasks.

1. Define a `ComputeTask` class with the following attributes:
 - `task_id` (int)
 - `operation` (str) — e.g., `"square"`, `"cube"`, `"factorial"`
 - `input_value` (int)
 - `result` (initially `None`)
 - A method `execute()` that computes the result based on the operation and stores it in `self.result`.
 - A `__repr__` method for clean printing.
2. Create a list of 5 `ComputeTask` objects with different operations and input values.
3. Execute **only the first 2 tasks** (call `.execute()` on them).
4. **Pickle** the entire list to a file `task_queue.pkl`.
5. **Unpickle** the list from the file and print all tasks — verify that:
 - The first 2 tasks show their computed results.
 - The remaining 3 tasks still show `result=None`.
6. Execute the remaining tasks on the unpickled list and print the final state.

Hints & Guidelines

- For `"factorial"`, use `math.factorial()` from the standard library.
- Remember: pickle preserves the **exact state** of objects — completed tasks remain completed, pending tasks remain pending.

- The class definition **must exist** when you unpickle — in this exercise, pickling and unpickling happen in the same script, so this is automatic.

Best Practices

- Use `"wb"` mode for writing pickle files and `"rb"` for reading — pickle is a **binary** format.
- Never unpickle files from untrusted sources in real applications.
- Clean up temporary files at the end of your program using `os.remove()`.

Exercise 3: Parallel File Word Counter

Topic: Multiprocessing — Process and Pool

Problem Statement

Write a program `ex03_parallel_word_counter.py` that counts words in multiple text files using both sequential and parallel approaches.

1. **Generate test data:** Create 6 text files (`file_0.txt` through `file_5.txt`), each containing 5,000–10,000 random words. You can generate random words by repeating a list of sample words:

```
import random
words = ["python", "parallel", "computing", "process", "thread",
        "memory", "data", "algorithm", "performance", "speedup"]
content = " ".join(random.choices(words, k=8000))
```

2. Write a function `count_words(filepath)` that reads a file and returns a tuple (`filepath`, `word_count`).
3. **Sequential run:** Call `count_words()` on each file in a loop and measure the time.
4. **Parallel run:** Use `multiprocessing.Pool.map()` to count words across all files simultaneously. Measure the time.
5. Print the results from both runs (they should be identical) and calculate the speedup.
6. **Clean up** — delete the generated test files.

Hints & Guidelines

- Use `len(content.split())` to count words in a string.
- Use `time.time()` before and after each run to measure elapsed time.
- The files are small, so the speedup may be modest or even negative due to process creation overhead. That's an important observation in itself!
- To make the parallel version show a speedup, add a small artificial delay in `count_words()` using `time.sleep(0.5)` to simulate reading from a slow disk or network drive.

Best Practices

- Always guard your multiprocessing code with `if __name__ == "__main__":`.
 - Define worker functions at **module level** (not inside `main()` or inside another function) — this is required on macOS and Windows.
 - Use `with Pool(...) as pool:` for automatic cleanup of worker processes.
-

Exercise 4: Matrix Operations with Threads

Topic: Multithreading — Basics and Shared Memory

Problem Statement

Write a program `ex04_threaded_matrix_ops.py` that performs element-wise operations on matrices using threads.

1. Create two 5×5 matrices (use nested lists or any structure you prefer):

```
matrix_a = [[random.randint(1, 10) for _ in range(5)] for _ in range(5)]
matrix_b = [[random.randint(1, 10) for _ in range(5)] for _ in range(5)]
```

2. Create a shared result matrix initialized to zeros:

```
result = [[0] * 5 for _ in range(5)]
```

3. Write a function `add_row(a, b, result, row_index)` that computes the element-wise sum of row `row_index` of matrices `a` and `b`, and stores the result in `result[row_index]`.
4. Create **one thread per row** (5 threads total) that each computes one row of the result matrix.
5. Start all threads, join them, and print the result matrix.
6. Verify correctness by computing the same result sequentially and comparing.

Hints & Guidelines

- Since threads share memory, all threads can read from `matrix_a` and `matrix_b` and write to different rows of `result` **without locks** — because each thread writes to a **different row** (no overlap).
- If threads were writing to the **same** row or variable, you would need a `threading.Lock`.
- Use `threading.Thread(target=add_row, args=(matrix_a, matrix_b, result, i))` for each row `i`.

Best Practices

- When threads write to non-overlapping regions of shared data, locks are unnecessary and would only slow things down.

- Always `join()` all threads before reading the result — otherwise you may read incomplete data.
- Print both the expected and actual result to visually verify correctness.

Exercise 5: Producer–Consumer Pipeline

Topic: Thread Communication and Queues

Problem Statement

Write a program `ex05_pipeline.py` that implements a **three-stage processing pipeline** using threads and queues:

```
[Generator] → Queue 1 → [Processor] → Queue 2 → [Writer]
```

1. **Generator thread:** Produces 20 "raw data" items. Each item is a dictionary:

```
{"id": i, "value": random.randint(1, 100)}
```

Put each item into `queue_1` with a small delay (0.1s).

2. **Processor thread:** Reads items from `queue_1`, transforms each item by squaring the `"value"` field, and puts the transformed item into `queue_2`.
3. **Writer thread:** Reads transformed items from `queue_2` and collects them into a list. After all items are processed, prints the final list.
4. Use `None` as a sentinel value to signal shutdown through the pipeline:
 - Generator sends `None` into `queue_1` when done.
 - Processor passes `None` into `queue_2` when it receives `None`.
 - Writer stops when it receives `None`.
5. Start all three threads, join them, and print a summary.

Hints & Guidelines

- Use `queue.Queue()` for both queues — they are thread-safe.
- Each stage should run in its own function, launched as a `threading.Thread`.
- The sentinel `None` propagates through the pipeline, cleanly shutting down each stage in order.
- Print a message in each stage (e.g., `[Generator] produced item 5`) so you can observe the interleaved pipeline execution.

Best Practices

- Bounded queues (`Queue(maxsize=5)`) apply back-pressure — if the processor is slow, the generator blocks instead of flooding memory.

- The pipeline pattern is fundamental — it appears in video processing, ETL (extract-transform-load), log processing, and many HPC workflows.
- Keep each stage simple and focused — a clean pipeline is easy to debug and extend.

Exercise 6: GIL Impact Experiment

Topic: Global Interpreter Lock — CPU-Bound vs I/O-Bound

Problem Statement

Write a program `ex06_gil_experiment.py` that empirically demonstrates the GIL's impact by running the **same computation** with varying numbers of threads and processes.

1. Define a CPU-bound function `fibonacci(n)` that computes the nth Fibonacci number using a naive recursive approach (intentionally slow):

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

2. Define an I/O-bound function `download_simulation(url_id)` that simulates downloading data:

```
def download_simulation(url_id):  
    time.sleep(random.uniform(0.5, 1.5)) # simulated network delay  
    return f"data_from_{url_id}"
```

3. Write a `benchmark(func, args_list, mode)` function where `mode` is one of `"sequential"`, `"threaded"`, or `"multiprocess"`. It should:

- Run `func` for each item in `args_list` using the specified mode.
- Return the elapsed time.

4. Run the benchmark for **both** functions across all three modes:

- CPU-bound: `fibonacci(30)` repeated 4 times
- I/O-bound: `download_simulation(i)` for `i` in `range(8)`

5. Print a clear comparison table showing times and speedups.

Hints & Guidelines

- `fibonacci(30)` takes roughly 0.2–0.5 seconds — long enough to see the GIL effect, short enough to finish quickly.
- For threaded mode, create `threading.Thread` objects.
- For multiprocess mode, create `multiprocessing.Process` objects.
- You should observe:

- CPU-bound: threads $\approx$ sequential, processes much faster.
- I/O-bound: threads $\approx$ processes, both much faster than sequential.

Best Practices

- Run the sequential version first to establish a **baseline** — all speedup numbers are relative to this baseline.
 - Use `time.perf_counter()` for more precise timing than `time.time()`.
 - Wrap multiprocessing code in `if __name__ == "__main__":`.
-

Exercise 7: Race Condition Detector

Topic: Sharing State Between Threads

Problem Statement

Write a program `ex07_race_detector.py` that demonstrates, detects, and fixes a race condition.

1. Part A — Observe the race condition:

- Create a shared dictionary:

```
bank_account = {"balance": 1000}
```

- Write a function `withdraw(account, amount, num_times)` that performs `num_times` withdrawals of `amount` from the account:

```
for _ in range(num_times):
    if account["balance"] >= amount:
        current = account["balance"]
        time.sleep(0.0001) # simulate processing delay
        account["balance"] = current - amount
```

- Launch 5 threads, each performing 100 withdrawals of \$1.
- After all threads finish, print the final balance.
- **Expected** balance: $\$1000 - (5 \times 100 \times \$1) = \$500$.
- **Actual** balance will likely be wrong — this is the race condition!

2. Part B — Fix with a Lock:

- Repeat Part A, but protect the withdrawal logic with a `threading.Lock`.
- Verify the final balance is exactly \$500.

3. Part C — Run the experiment 10 times:

- Run both the unsafe and safe versions 10 times each.
- Print how many times each version produced the correct result.

- The unsafe version should fail frequently; the safe version should always succeed.

Hints & Guidelines

- The `time.sleep(0.0001)` between reading and writing the balance is crucial — it widens the race window so the bug manifests reliably.
- Without the sleep, CPython's GIL may accidentally serialize the operations, masking the bug — but **the bug is still there** and would surface under different conditions or implementations.
- Use `with lock:` (context manager syntax) instead of manual `lock.acquire()` / `lock.release()` — it's safer and cleaner.

Best Practices

- **Assume every shared mutation is a race** until you've protected it with a lock, semaphore, or atomic operation.
- Keep the critical section (code inside `with lock:`) as **short as possible** — hold the lock only while accessing shared data.
- Consider using `threading.RLock` (reentrant lock) if the same thread might need to acquire the lock multiple times (nested calls).

Exercise 8: Process vs Thread Memory Isolation Lab

Topic: Sharing State Between Processes

Problem Statement

Write a program `ex08_memory_isolation.py` that explores memory isolation between processes and compares different IPC (Inter-Process Communication) mechanisms.

1. Part A — Demonstrate isolation:

- Create a global list `shared_log = []`.
- Write a function `log_message(msg)` that appends `msg` to `shared_log` and prints the list's length inside the child.
- Spawn 3 processes, each calling `log_message()` with a unique message.
- After all processes finish, print `shared_log` from the parent.
- **Observe:** The parent's list is empty — each child has its own copy.

2. Part B — Share data using `multiprocessing.Array`:

- Create a shared array of 10 integers, initialized to 0:

```
shared_array = multiprocessing.Array('i', 10)
```

- Write a function `fill_range(arr, start, end, value)` that fills `arr[start:end]` with `value`.
- Spawn 2 processes:
 - Process 1 fills indices 0–4 with the value `1`.

- Process 2 fills indices 5–9 with the value 2.
- After both finish, print the array from the parent: `[1, 1, 1, 1, 1, 2, 2, 2, 2, 2]`.

3. Part C — Share data using `multiprocessing.Queue`:

- Create a `multiprocessing.Queue`.
- Spawn 4 worker processes, each computing the square of a different number and putting the result (`number`, `square`) into the queue.
- In the parent, read all 4 results from the queue and print them.

Hints & Guidelines

- `multiprocessing.Array('i', 10)` creates a shared C-style integer array of 10 elements. Access elements with `arr[i]`.
- `multiprocessing.Queue` works like `queue.Queue` but is safe for use between processes (not just threads).
- `Queue.get()` blocks until an item is available. If you know exactly how many items to expect, call `get()` that many times.
- All functions used as process targets **must** be at module level.

Best Practices

- Prefer `Queue` or `Pipe` for message passing between processes — they are simpler and safer than shared memory.
- Use shared memory (`Value`, `Array`) only when performance is critical and you need to avoid the overhead of serializing/deserializing data.
- Always use locks when multiple processes write to overlapping regions of shared memory.

Exercise 9: Parallel Speedup Analyzer

Topic: Performance Evaluation — Speedup, Efficiency, Amdahl's Law

Problem Statement

Write a program `ex09_speedup_analyzer.py` that measures and visualizes the parallel performance characteristics of a workload.

1. **Choose a workload:** Implement a function `monte_carlo_pi(num_samples)` that estimates π using the Monte Carlo method:

```
import random
def monte_carlo_pi(num_samples):
    inside = 0
    for _ in range(num_samples):
        x = random.random()
        y = random.random()
        if x*x + y*y <= 1.0:
```

```
        inside += 1
    return inside
```

The estimate of π is: `4 * total_inside / total_samples`.

2. **Sequential baseline:** Run `monte_carlo_pi(5_000_000)` and measure the time.
3. **Parallel runs:** Split `5_000_000` samples across N workers using `multiprocessing.Pool.map()`. Each worker runs `monte_carlo_pi(5_000_000 // N)`. Sum the `inside` counts and compute the π estimate.

Run for N = 1, 2, 4, and up to your machine's core count.

4. **Compute and print** for each N:

- Parallel time
- Speedup = $T_{\text{seq}} / T_{\text{par}}$
- Efficiency = Speedup / N
- Amdahl's theoretical speedup (assume serial fraction = 3%)
- Gustafson's scaled speedup

5. **Print the results** as a formatted table:

Workers	Time(s)	Speedup	Efficiency	Amdahl	Gustafson
1	2.450	1.00	100.0%	1.00	1.00
2	1.280	1.91	95.6%	1.94	1.97
4	0.670	3.66	91.4%	3.66	3.91
8	0.380	6.45	80.6%	6.31	7.79

6. Also print the **π estimate** from each run — it should converge near `3.14159`.

Hints & Guidelines

- Each worker should use `random.random()` — the `random` module automatically seeds differently in each process, so results won't be identical (which is actually what we want for Monte Carlo).
- The function should return the count of points **inside** the circle, not the π estimate itself — the parent process sums the counts and computes π .
- Amdahl's formula: $1 / (S + (1-S)/N)$ where S is the serial fraction.
- Gustafson's formula: $N - S \times (N - 1)$.

Best Practices

- Monte Carlo methods are **embarrassingly parallel** — each sample is independent of every other sample. This is the ideal case for `Pool.map()`.
- When comparing timings, run each configuration **at least 3 times** and use the average to reduce noise from OS scheduling and background processes.

- Print intermediate results (the π estimate) to verify that correctness is maintained as you parallelize — speedup is meaningless if the answer is wrong.

Exercise 10: Build Your Own Parallel Toolkit 🏗️

Topic: Integrating all Day 1 concepts

Problem Statement

Write a program `ex10_parallel_toolkit.py` that combines serialization, multiprocessing, threading, and performance measurement into a single cohesive tool.

Scenario: You are building a data processing pipeline that:

1. **Loads** a list of tasks from a JSON configuration file.
2. **Processes** the tasks in parallel using a process pool.
3. **Saves** the results using Pickle.
4. **Reports** performance metrics.

Step-by-Step

Step 1 — Create the task file:

Create a file `tasks.json` containing:

```
[
    {"id": 1, "operation": "sum_of_squares", "n": 2000000},
    {"id": 2, "operation": "sum_of_squares", "n": 3000000},
    {"id": 3, "operation": "sum_of_squares", "n": 1500000},
    {"id": 4, "operation": "sum_of_squares", "n": 2500000},
    {"id": 5, "operation": "sum_of_squares", "n": 1000000},
    {"id": 6, "operation": "sum_of_squares", "n": 3500000}
]
```

Step 2 — Load tasks:

Read the JSON file and parse it into a list of task dictionaries.

Step 3 — Define the worker function:

```
def execute_task(task):
    """Execute a single task and return the result."""
    start = time.time()
    # Compute sum of squares from 0 to task["n"]
    result = sum(i * i for i in range(task["n"]))
    elapsed = time.time() - start
    return {
        "id": task["id"],
        "n": task["n"],
```

```
"result": result,
"time": round(elapsed, 4)
}
```

Step 4 — Run sequentially and in parallel:

- Sequential: `[execute_task(t) for t in tasks]`
- Parallel: `pool.map(execute_task, tasks)`

Step 5 — Save results:

Pickle the list of result dictionaries to `results.pkl`.

Step 6 — Report:

Print a table showing:

- Each task's ID, `n`, result, and execution time
- Total sequential time vs parallel time
- Speedup and efficiency

Hints & Guidelines

- This exercise ties together JSON (config loading), Pickle (result persistence), multiprocessing (parallel execution), and performance measurement — all topics from Day 1.
- Start by getting the sequential version working, then add parallelism.
- Use `json.load()` for input and `pickle.dump()` for output — this mirrors a common real-world pattern (human-readable config in, machine-efficient results out).

Best Practices

- Separate concerns: loading, processing, saving, and reporting should each be their own function.
- Validate the JSON structure before processing — check that required keys exist.
- Print the speedup **only after verifying** that sequential and parallel results match.

General Tips for All Exercises

1. **Start simple** — get a basic version working before adding complexity.
 2. **Print intermediate values** — when debugging parallel code, print statements with PID/TID help you trace what's happening.
 3. **Always guard multiprocessing** — use `if __name__ == "__main__":` in every script that uses `multiprocessing`.
 4. **Module-level functions** — on macOS and Windows, functions passed to `Process()` or `Pool()` must be defined at the top level of the module, not inside `main()` or another function.
 5. **Compare results** — always verify that the parallel version produces the same result as the sequential version before celebrating the speedup.
 6. **Measure, don't guess** — use `time.time()` or `time.perf_counter()` to measure actual performance. Intuition about what's "fast" is often wrong.
-

Happy coding! These exercises build on each other — concepts from earlier exercises will help you solve later ones.